

Assignment 8.3

M.koushik reddy

2103A52153

Task 1

```
def is_valid_email(email):  
    # Must contain exactly one @  
    if email.count("@") != 1:  
        return False  
  
    # Must contain .  
    if "." not in email:  
        return False  
  
    # Must not start or end with special characters  
    special_chars = "@._"  
    if email[0] in special_chars or email[-1] in special_chars:  
        return False  
  
    return True  
  
# AI-generated test cases  
def run_tests():  
    test_cases = [  
        ("test@gmail.com", True),  
        ("user.name@domain.com", True),  
        ("hello@abc.in", True),  
        ("@gmail.com", False),  
        ("testgmail.com", False),  
        ("test@@gmail.com", False),  
        ("test@gmailcom", False),  
        ("test@gmail.com.", False),  
        (".test@gmail.com", False),  
    ]
```

```

for email, expected in test_cases:
    result = is_valid_email(email)
    print(f"{email} -> {result} (Expected: {expected})")

run_tests()

```

Output

```

task 1.py"
test@gmail.com -> True (Expected: True)
user.name@domain.com -> True (Expected: True)
hello@abc.in -> True (Expected: True)
@gmail.com -> False (Expected: False)
testgmail.com -> False (Expected: False)
test@gmail.com -> False (Expected: False)
test@gmailcom -> False (Expected: False)
test@gmail.com. -> False (Expected: False)
.test@gmail.com -> False (Expected: False)
PS C:\Users\koushik reddy\Desktop\python practice> █

```

Task 2

```

def assign_grade(score):
    # Handle invalid inputs
    if not isinstance(score, (int, float)):
        return "Invalid input"
    if score < 0 or score > 100:
        return "Invalid input"

    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"

```

```
    else:
        return "F"

# AI-generated test cases using loop
def run_tests():
    test_cases = [
        (95, "A"),
        (90, "A"),    # boundary
        (85, "B"),
        (80, "B"),    # boundary
        (75, "C"),
        (70, "C"),    # boundary
        (65, "D"),
        (60, "D"),    # boundary
        (40, "F"),
        (-5, "Invalid input"),
        (105, "Invalid input"),
        ("eighty", "Invalid input"),
    ]

    for score, expected in test_cases:
        result = assign_grade(score)
        print(f"Score: {score} → Grade: {result} (Expected: {expected})")

run_tests()
```

Output

```

PS C:\Users\koushik reddy\Desktop\python practice> & "C:/Users/
task 1.py"
Score: 95 → Grade: A (Expected: A)
Score: 90 → Grade: A (Expected: A)
Score: 85 → Grade: B (Expected: B)
Score: 80 → Grade: B (Expected: B)
Score: 75 → Grade: C (Expected: C)
Score: 70 → Grade: C (Expected: C)
Score: 65 → Grade: D (Expected: D)
Score: 60 → Grade: D (Expected: D)
Score: 40 → Grade: F (Expected: F)
Score: -5 → Grade: Invalid input (Expected: Invalid input)
Score: 105 → Grade: Invalid input (Expected: Invalid input)
Score: eighty → Grade: Invalid input (Expected: Invalid input)
PS C:\Users\koushik reddy\Desktop\python practice> █

```

Task 3

```
import string
```

```

def is_sentence_palindrome(sentence):
    # Remove spaces, punctuation, and convert to lowercase
    cleaned = ""
    for ch in sentence.lower():
        if ch.isalnum():
            cleaned += ch

    return cleaned == cleaned[::-1]

# AI-generated test cases
def run_tests():
    test_cases = [
        ("A man a plan a canal Panama", True),
        ("Madam", True),
        ("Was it a rat I saw?", True),
        ("Hello world", False),
        ("Python", False),
        ("No lemon, no melon", True),
    ]

    for sentence, expected in test_cases:

```

```
result = is_sentence_palindrome(sentence)
print(f"{sentence}" -> {result} (Expected: {expected}))

run_tests()
```

Output

```
task 1.py
"A man a plan a canal Panama" -> True (Expected: True)
"Madam" -> True (Expected: True)
"Was it a rat I saw?" -> True (Expected: True)
"Hello world" -> False (Expected: False)
"Python" -> False (Expected: False)
"No lemon, no melon" -> True (Expected: True)
PS C:\Users\koushik reddy\Desktop\python practice> █
```

Task 4

```
class ShoppingCart:
    def __init__(self):
        self.items = {}

    def add_item(self, name, price):
        self.items[name] = price

    def remove_item(self, name):
        if name in self.items:
            del self.items[name]
        else:
            print("Item not found")

    def total_cost(self):
        return sum(self.items.values())
```

```

# AI-generated test cases
def run_tests():
    cart = ShoppingCart()

    # Empty cart test
    print("Empty cart total:", cart.total_cost()) # Expected: 0

    # Add items
    cart.add_item("Book", 250)
    cart.add_item("Pen", 50)
    print("Total after adding items:", cart.total_cost()) # Expected: 300

    # Remove item
    cart.remove_item("Pen")
    print("Total after removing Pen:", cart.total_cost()) # Expected: 250

    # Remove non-existing item
    cart.remove_item("Bag")

run_tests()

```

Output

```

Empty cart total: 0
Total after adding items: 300
Total after removing Pen: 250
Item not found
PS C:\Users\koushik reddy\Desktop\python practice>

```

Task 5

```

def convert_date_format(date_str):
    parts = date_str.split("-")

    if len(parts) != 3:

```

```

        return "Invalid format"

    year, month, day = parts
    return f"{day}-{month}-{year}"

# AI-generated test cases
def run_tests():
    test_cases = [
        ("2023-10-15", "15-10-2023"),
        ("2020-01-01", "01-01-2020"),
        ("1999-12-31", "31-12-1999"),
        ("2023/10/15", "Invalid format"),
        ("15-10-2023", "Invalid format"),
    ]

    for date_str, expected in test_cases:
        result = convert_date_format(date_str)
        print(f"{date_str} -> {result} (Expected: {expected})")

run_tests()

```

Output

```

task 1.py"
2023-10-15 -> 15-10-2023 (Expected: 15-10-2023)
2020-01-01 -> 01-01-2020 (Expected: 01-01-2020)
1999-12-31 -> 31-12-1999 (Expected: 31-12-1999)
2023/10/15 -> Invalid format (Expected: Invalid format)
15-10-2023 -> 2023-10-15 (Expected: Invalid format)
PS C:\Users\koushik reddy\Desktop\python practice> 

```